

8085 : Microprocessor Architecture, Programming & Applications by P.N. Ganesan. (8085 Microprocessor)

→ 8085 Micro → 8-bit, 3 Mega Hz, 14 pin, 40 pin, 85 instructions.

W, Z → Temporary 8 bit register
↓
for internal use.

B, C, D, E, H, L → general purpose register & all are 8-bit.
(1 flip flop - 1 bit & 8 flip flop - 1 register)

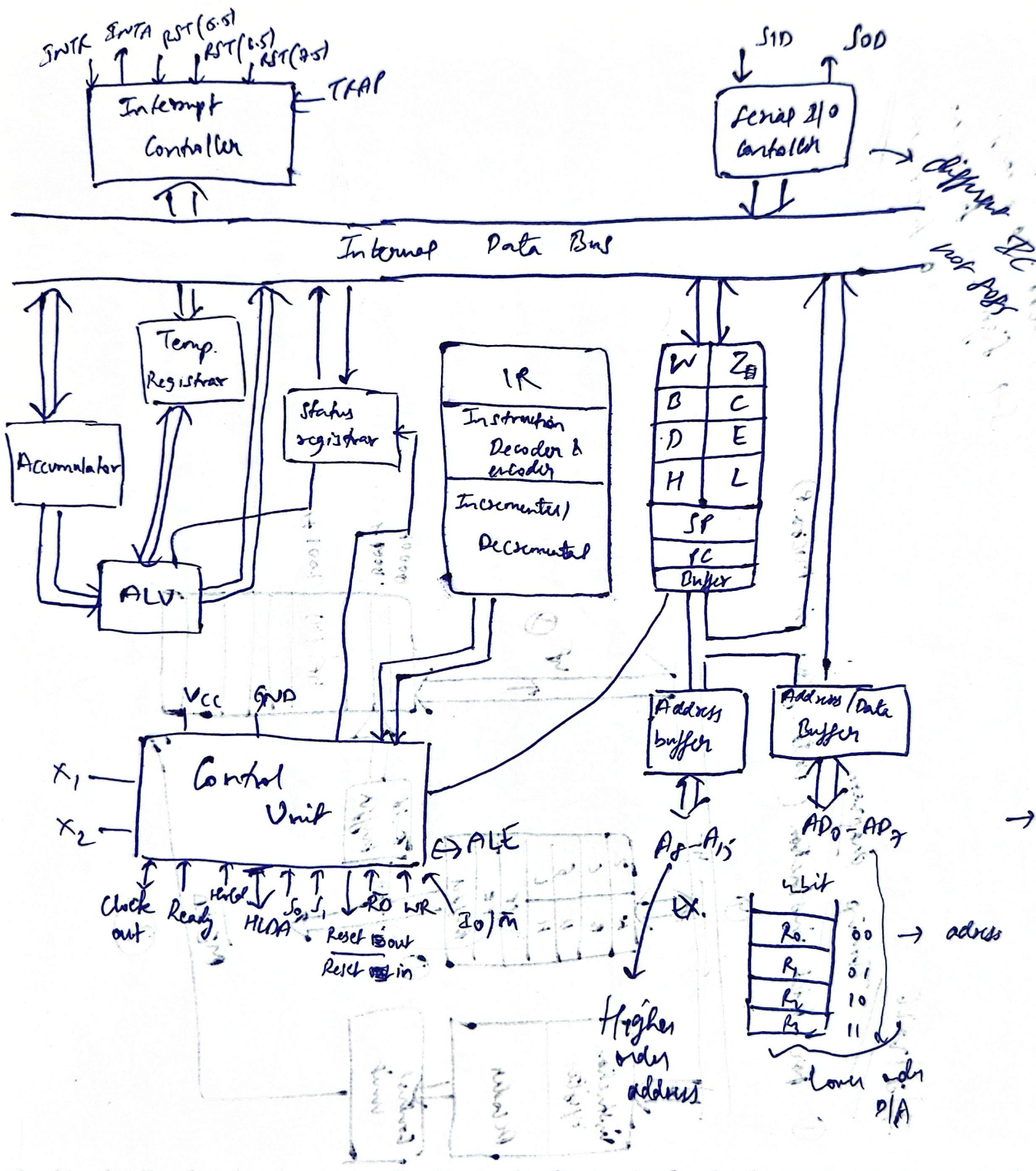
pair → BC, DE, HL for 16-bit pair

HL → used as memory pointer.

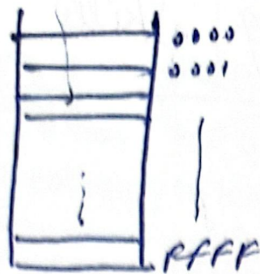
SP → 16-bit R → stack pointer

→ every register has an address

address limit $2^x = \text{bit}$



for 8085, 64 kbit,

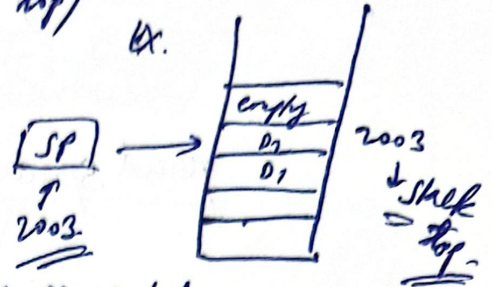


(AD₁₋₁₅ in diagram)
 16 address lines are needed
 $(2^{16} \approx 65536)$

Stack pointer has the address of ~~the~~ data piece. (stack top)

PC → program counter → 16 bit register

↳ holds the address of the next instruction to be executed.



Ex.

MVI	B, D ₁	D ₁ → B
(more immediately)		D ₂ → A
MVI	A, D ₂	A → A+B
ADD	B	
STA	XXXX (ex. 2001)	
HLT		

for each step, we do code 1, D₁
 code 2, D₂

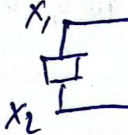
(program to add 2 bit without carry)

ex: 2000 → Code 1
 2001 → D₁
 2002 → Code 2
 2003 → D₂

→ Buffer → for avoiding loading

→ Address is needed Data identification & then execution of data. So there is

→ IR → Instruction register → gives the instruction like MVI, HLT, etc

→ X₁ & X₂ → clock signal (crystal oscillator)  to give clock pulse
 for (rectangular wave) form

The crystal oscillator generates X2 frequency so there is a divider inside control unit

ALE → latching signal

It is a very
interesting case of
a very

the value of the data given (the day)

the value of the data given

the value of the data given (the day)

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given (the day)

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

the value of the data given

2000 - 1000
2000 - 1000
2000 - 1000
2000 - 1000

big drawing

Only identification to the collection of data to the

the value of the data given (the day)

the value of the data given (the day)

the value of the data given (the day)

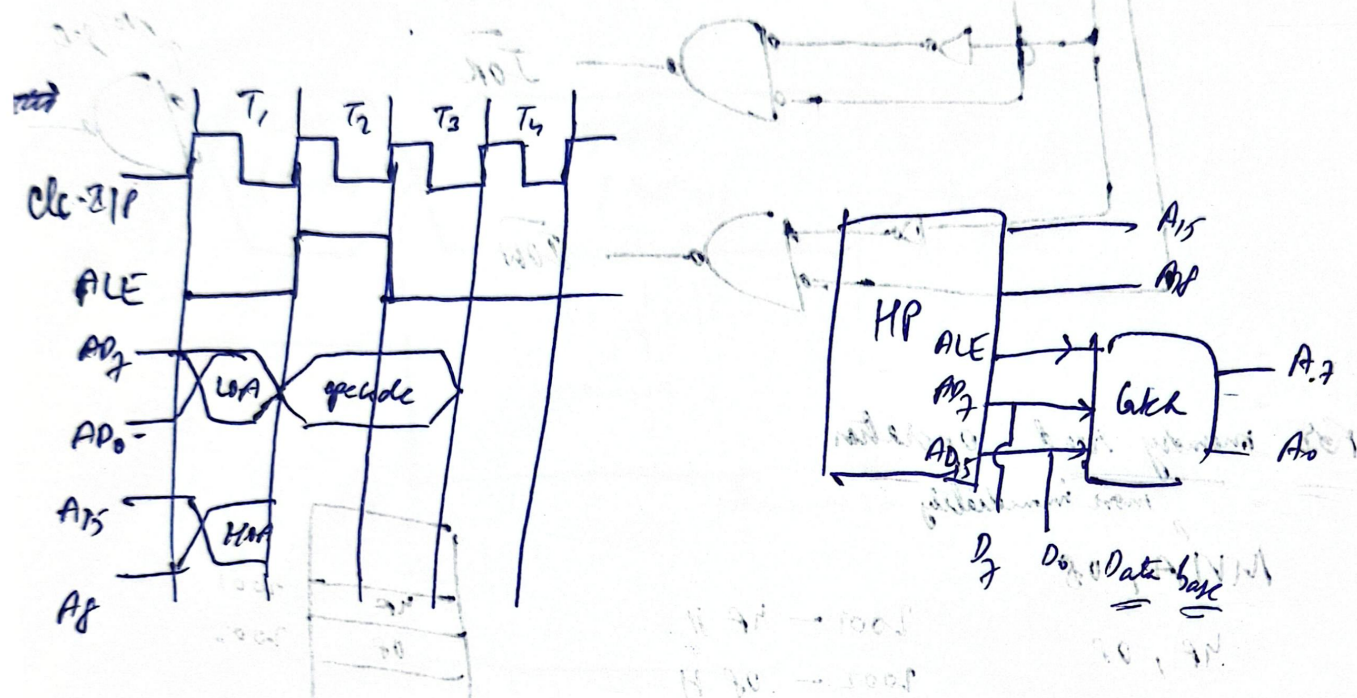
$I/O/M \rightarrow$ Whether the operation of MP is Input-output (I) Memory.

$\downarrow \downarrow$
2

$\rightarrow RD, WR \rightarrow$ Read & write $\rightarrow$ Combined for memory interface $\rightarrow IORD, IOWR / MEO, MWR$

$\rightarrow S_0, S_1 \rightarrow$ Status signal $\rightarrow$

S_0	S_1	
0	0	Standby
0	1	Write
1	0	Read
1	1	Fetch



$\rightarrow$ HOLD: Hold Acknowledge to acknowledge data & then holds the data.

$\rightarrow$ Ready: To indicate the readiness of MP

$\rightarrow$ Clk out: Generate clock for external device (purpose)

$\rightarrow$ SIO & SOD: Serial-I/O-Data : MP gives parallel data & serial data can be done, so this is for converting for data flow

$\rightarrow$ INTR $\rightarrow$ interrupt request $\rightarrow$ external device's signal
INTA $\rightarrow$ " acknowledge

$\rightarrow$ TRAP $\rightarrow$ Non maskable - can't disable

$\rightarrow$ RST $\rightarrow$ Restart interrupt $\rightarrow$ priority order $\rightarrow$ TRAP, 7.5, 6.5, 5.5, INTR

→ PSW independent Controller → ALU → Arithmetic Logic Unit

→ Temp. Reg → for Temp. purpose → 8 bit

→ Accumulator → for operation, always one data in accumulator → 8 bit

→ Status Register → 8 bit register → status of Arithmetic Logic output
(PSW)

P ₇	P ₆	P ₅	P ₄	P ₃	P ₂	P ₁	P ₀
S	Z	X	AC	X	P	X	C

C → Carry → flag = 1 if carry is there
0 if not there
X → undefined (0/1)

P → Parity → 21 for even parity
20 for odd "

AC → Auxiliary Carry

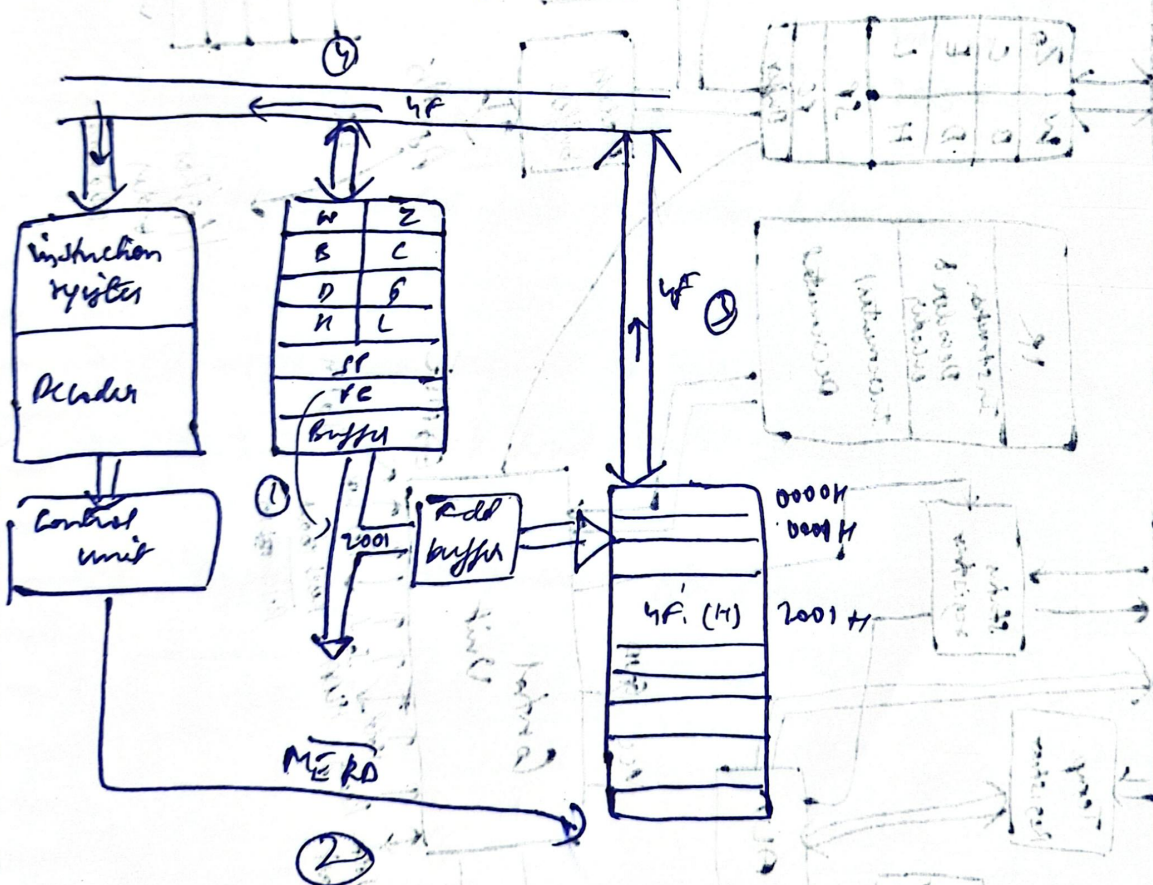
Z → Zero → 21 for zero result
20 for non zero "

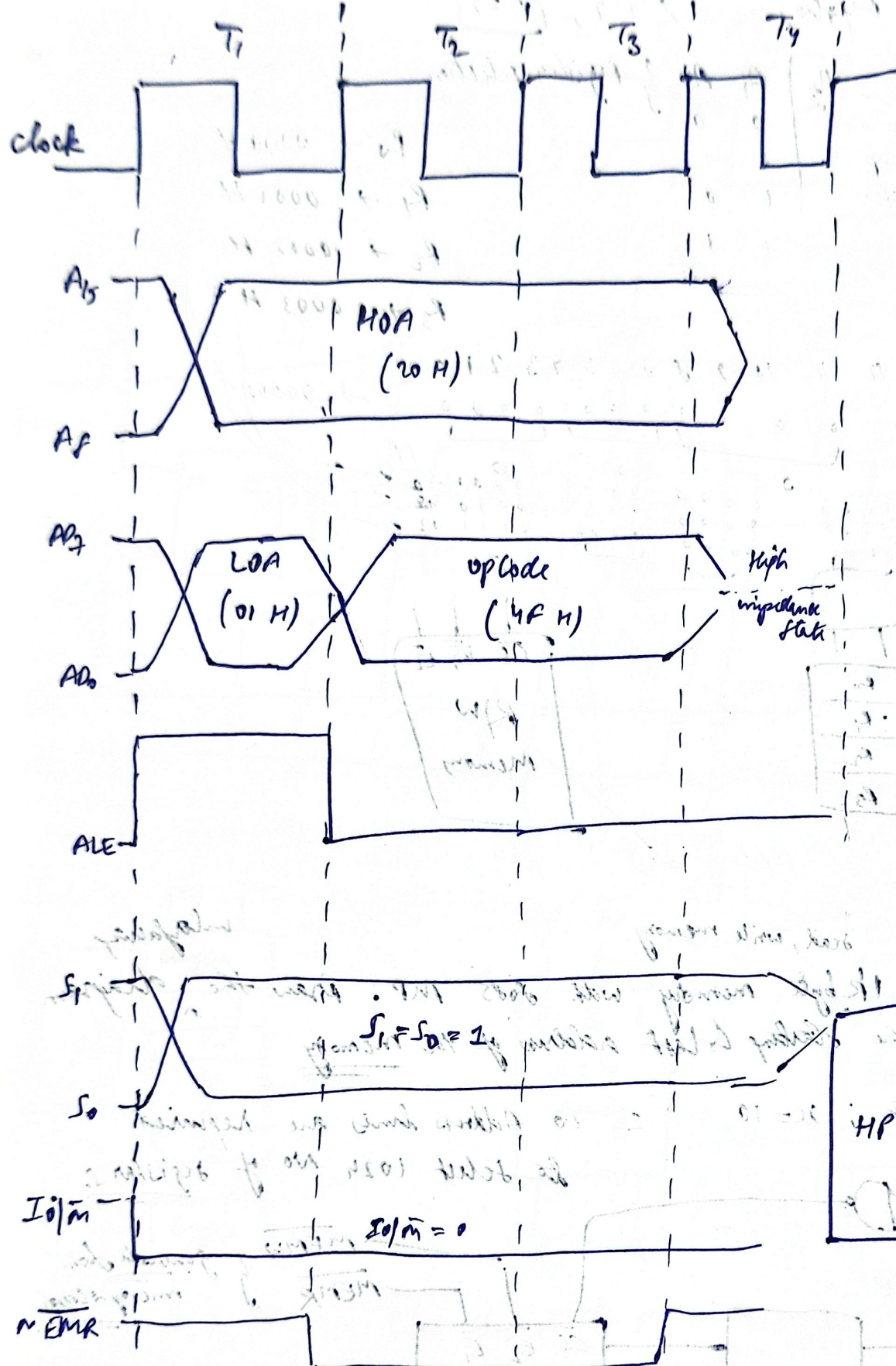
S → Sign → 1 for (pos) no
20 for (neg) no.

7/8/20.26

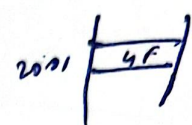
Timing diagram

① for full instruction (data flow in Micro.P)



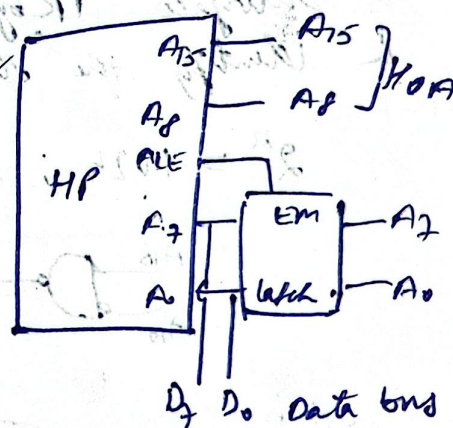


then is
4F opcode
in
2001 address



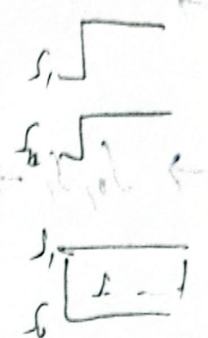
2001 → 4F
address opcode

high impedance state



- T_1 = identification of address
- T_2 T_3 = memory read operation
- T_4 = Decoding of instruction

10/10



② For memory read operation
= more immediately accumulator
MVA, 08

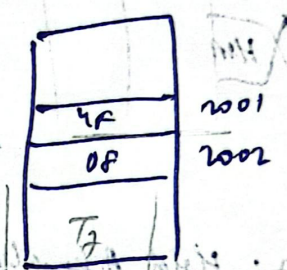
more immediately accumulator

MVA, 08

4f, 0p

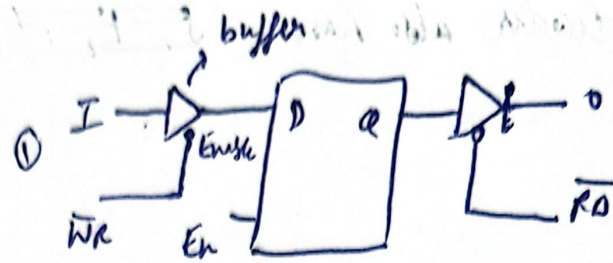
2001 - 4F H

2002 - of H



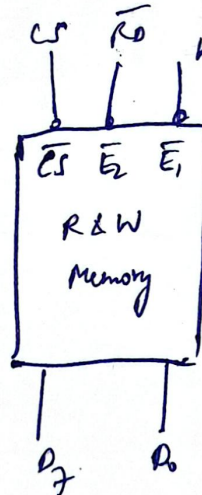
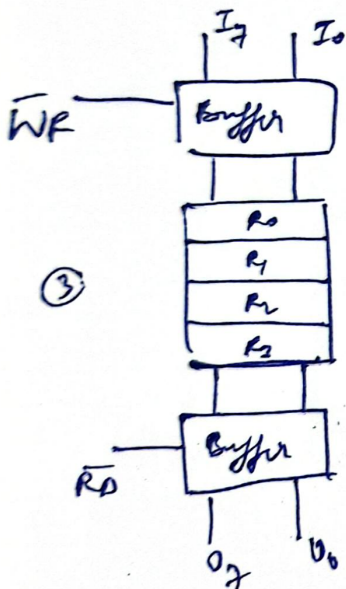
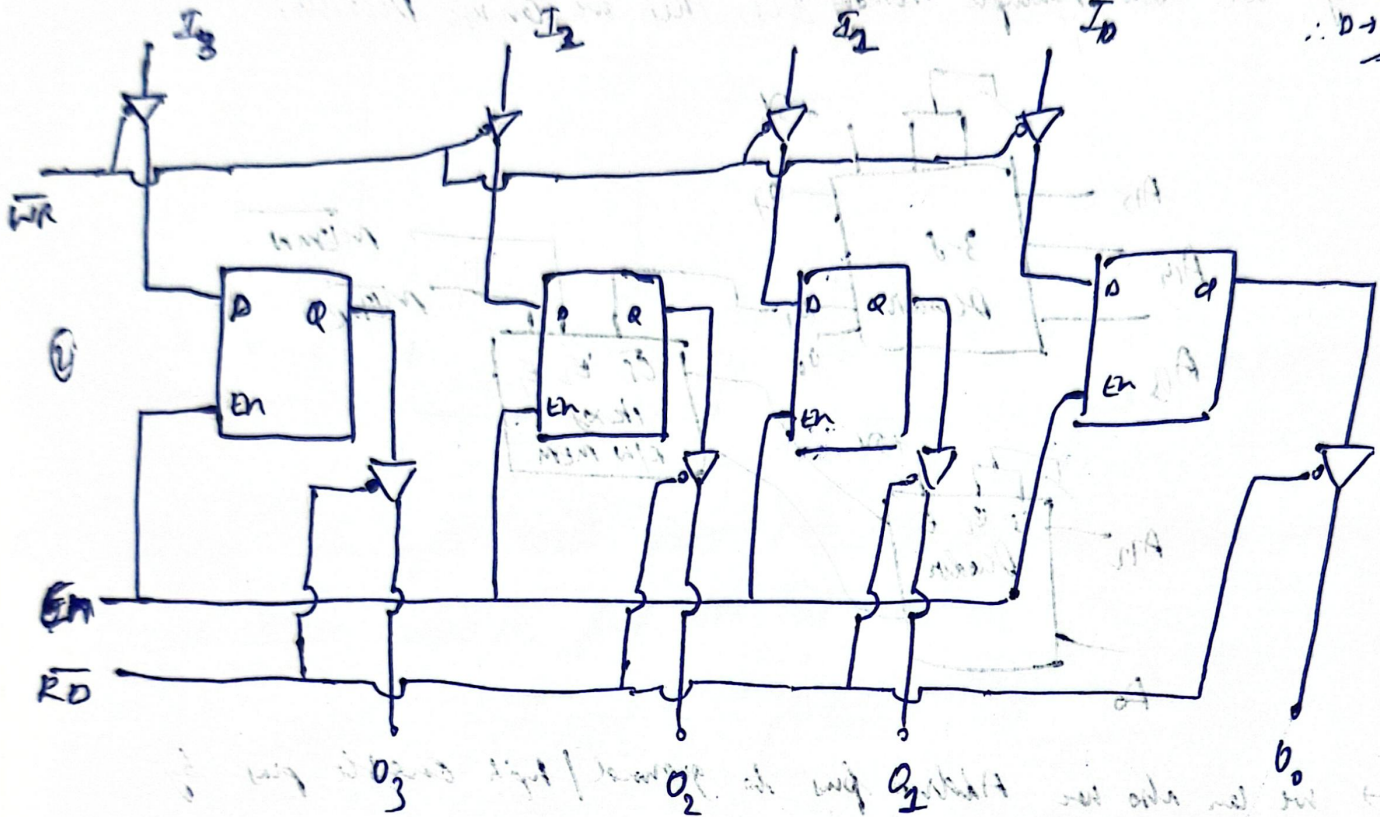
$\leftarrow \text{ATAC} \leftarrow$
 $\leftarrow \text{ATAC} \leftarrow$
 $\leftarrow \text{ATAC} \leftarrow$
 $\leftarrow \text{ATAC} \leftarrow$

Memory Interfacing



Enable = 0
I → 0, 0 → 1

E → 1
∴ D → 1

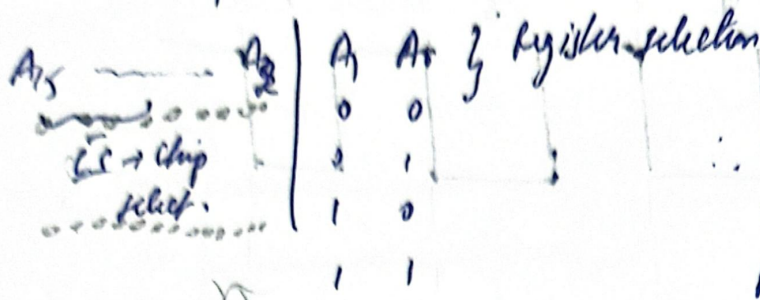


CS → chip selection for memory purposes

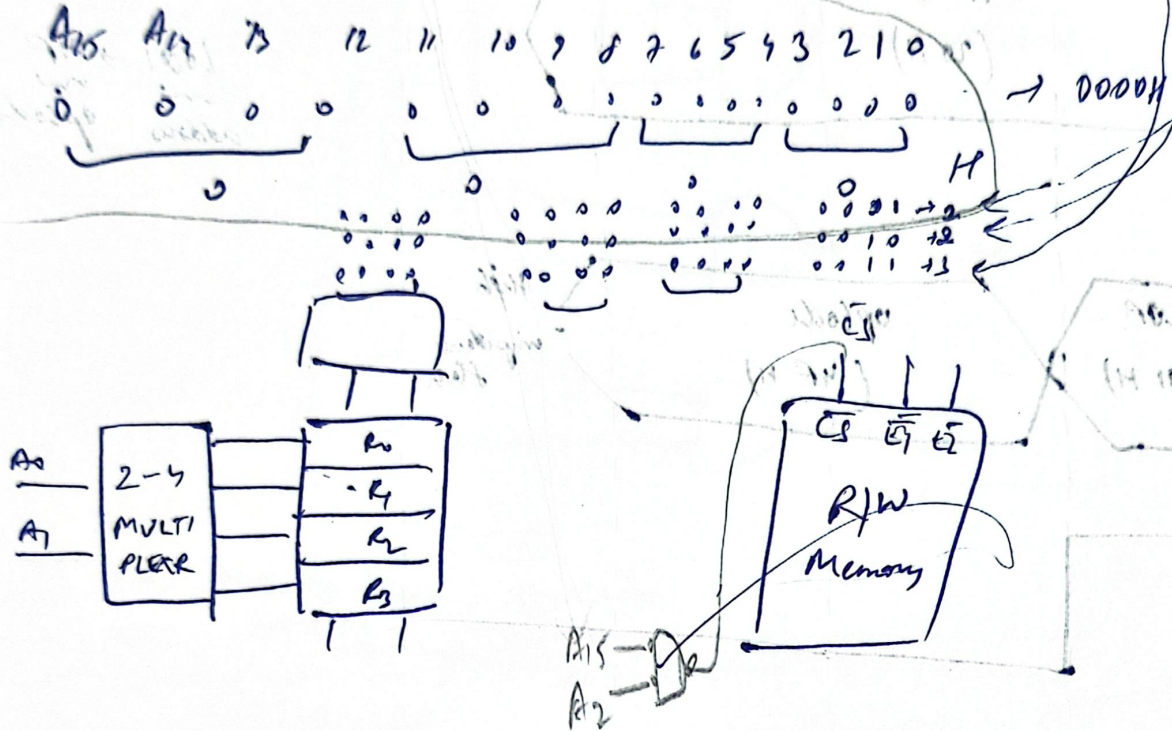
→ $2^x = \text{No of Registers}$, $x \rightarrow \text{No of address lines required}$

→ There shouldn't be memory pounding → same address line for different registers.

here we have 4 registers, $\therefore 2^2 = 4$, $(K=2)$



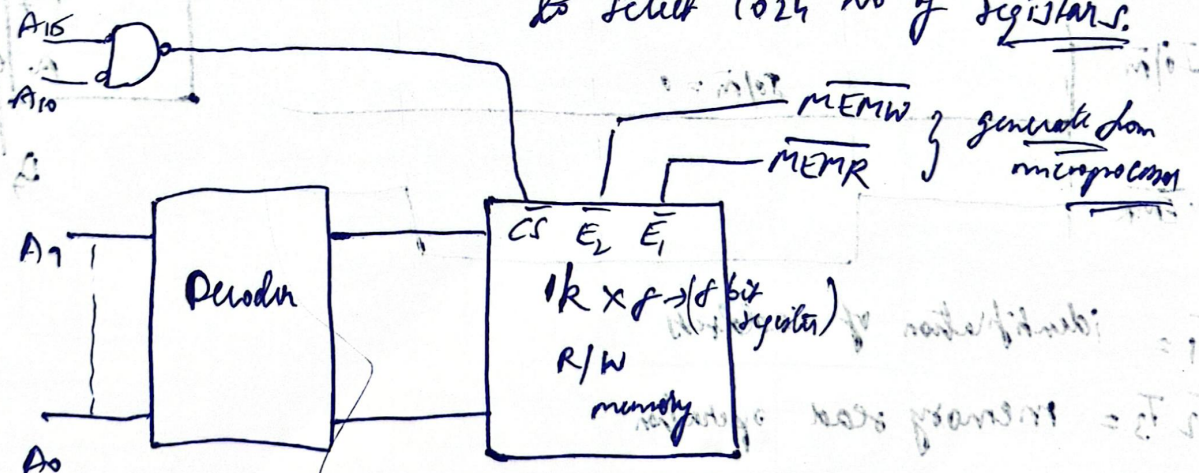
$R_0 \rightarrow 0000H$
 $R_1 \rightarrow 0001H$
 $R_2 \rightarrow 0002H$
 $R_3 \rightarrow 0003H$

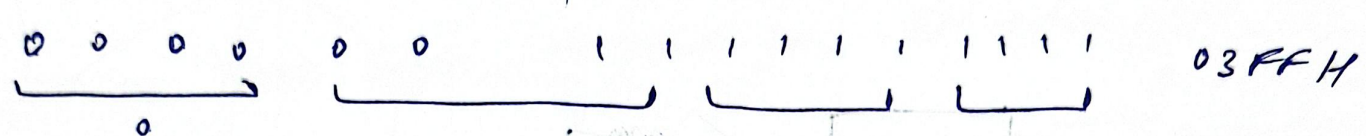
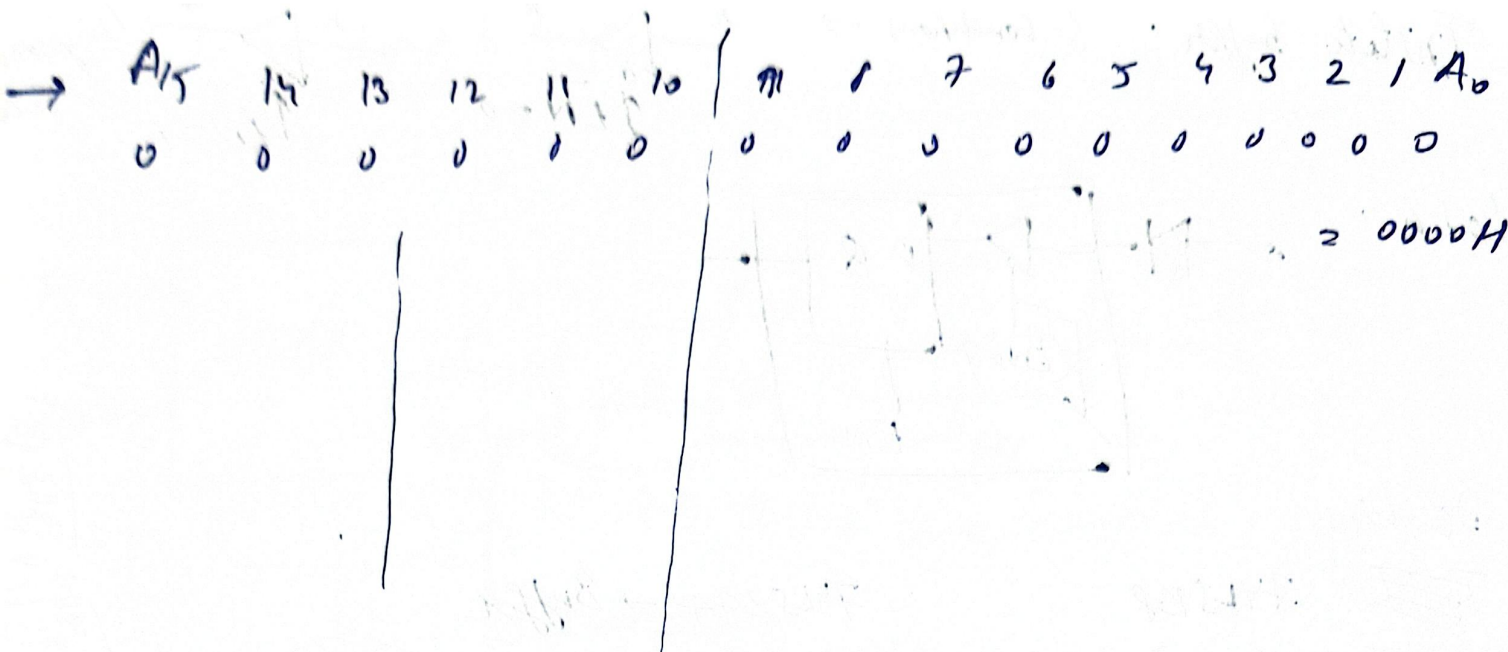


read, write memory

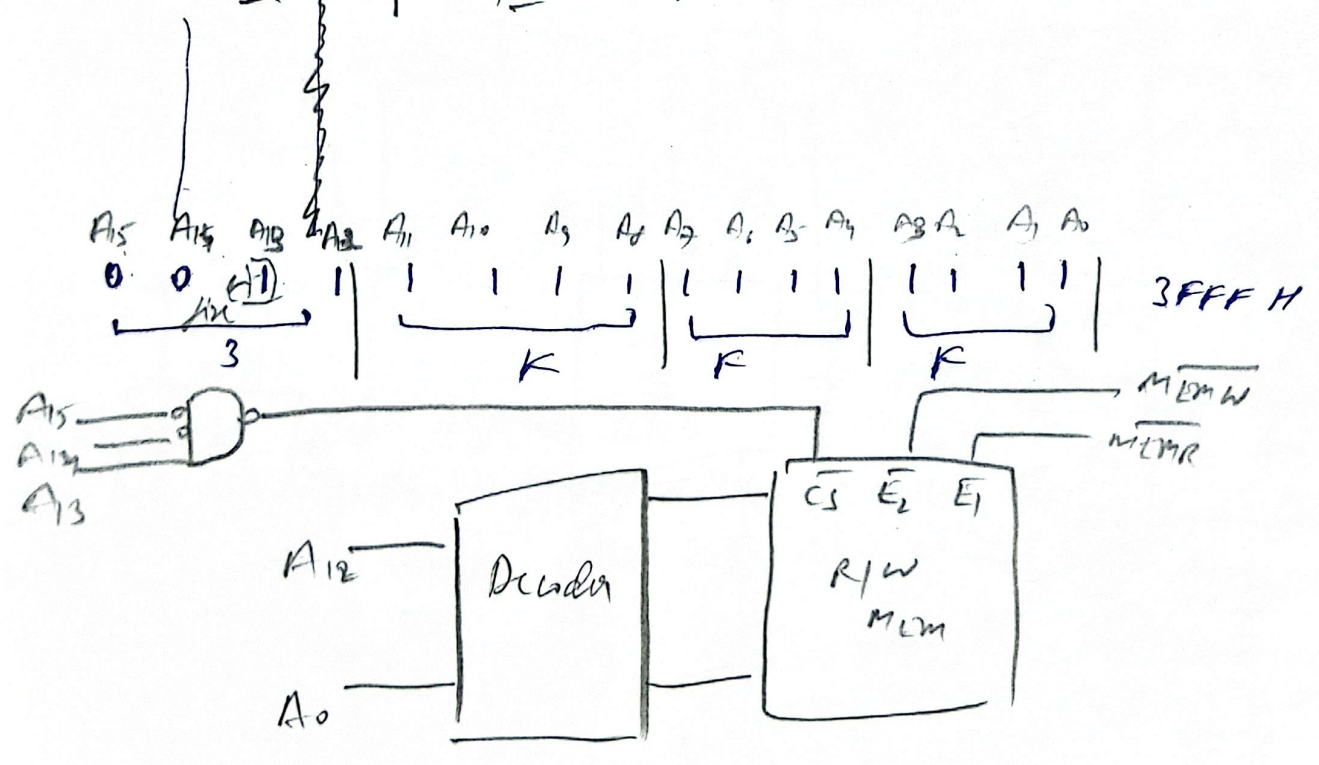
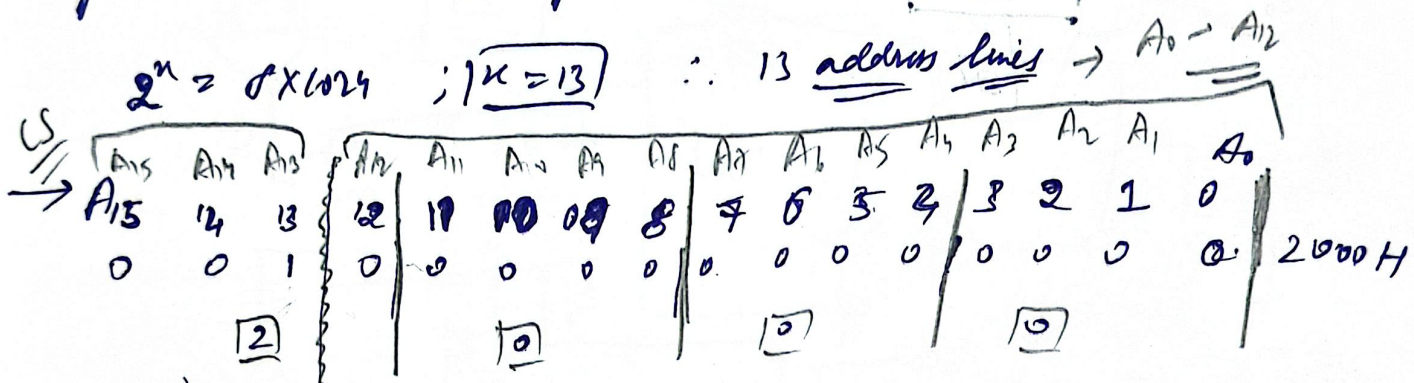
→ Interface 1K byte memory with 8085 MP. Draw the diagram & identify the starting & last address of the memory

$2^N = 1024$, $N=10$ $\therefore$ 10 Address lines are required to select 1024 No of registers.

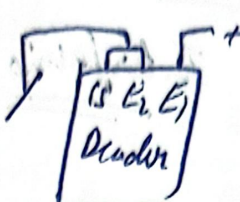




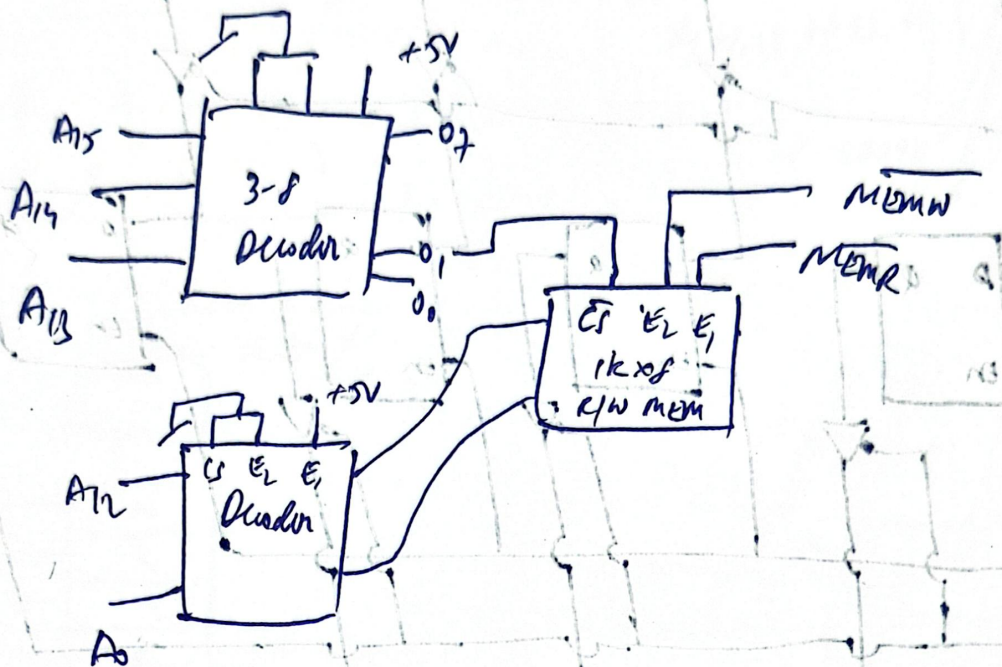
Q1. Interface 8K byte with 8085 MP. ~~Identify~~ the starting address of the Mem 2000H. find the last address.



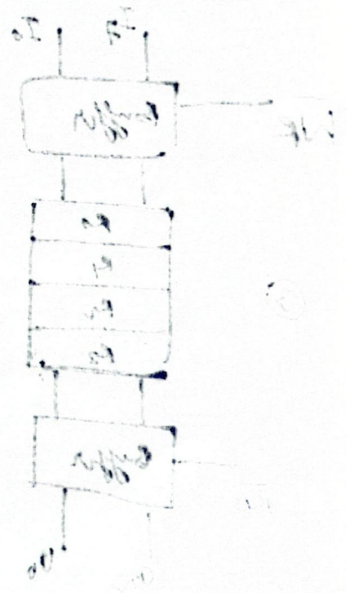
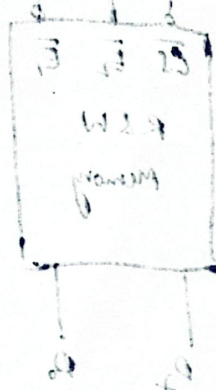
The decoder also have $(\overline{CS}, E_2, E_1)$ CS, E_2, E_1 & CS & E_2 should be low.
 $E_1 \rightarrow$ high



→ if we have multiple memory ICs, then we can use Decoder.



→ we can also use Address pins to ground / high enable pins by using appropriate logic gates:

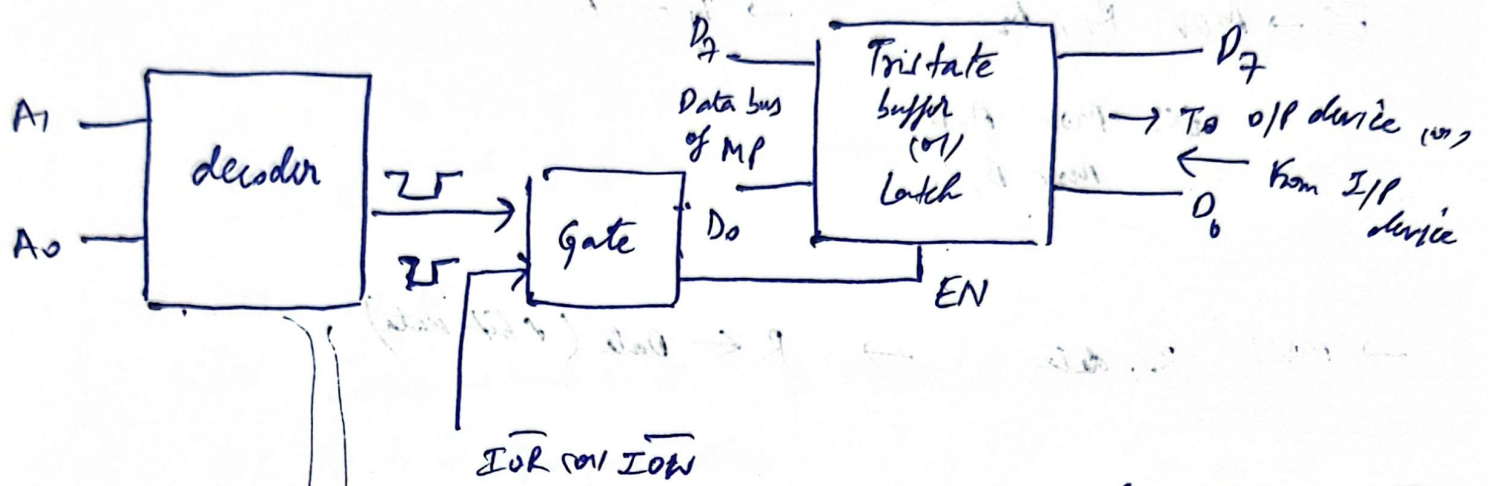


output given by address pins (0-7)

connected with address pins of all ICs, address pins of all

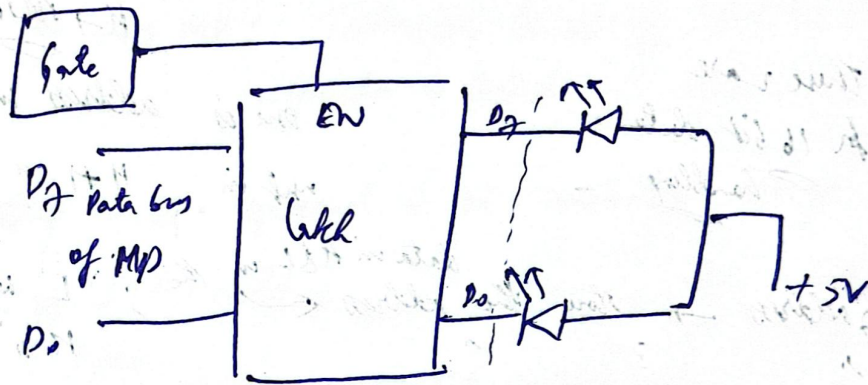
ICs of all address pins to ground / high enable pins of all ICs

Interfacing of Input & output

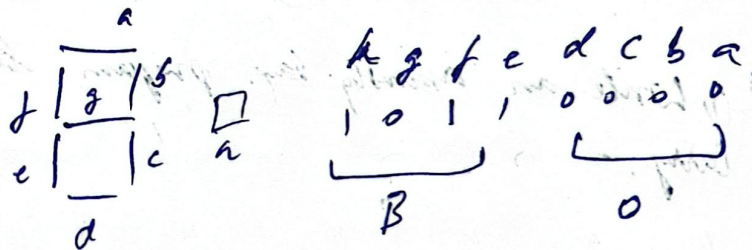


if we use Tristate buffer, we should use, $\overline{EN} \rightarrow$ low signal $\rightarrow$ to activate.

$\rightarrow$ for using latch circuit the $\overline{EN}$ signal should be high $\rightarrow$ to deactivate.



Let's say we need to glow 2,



$\rightarrow$ Interfacing seven segment LED with MP, also interface a group of switches with MP to transfer certain information to the output device.

Identify the I/O port address from the interfacing circuit.

MVI A, B0H

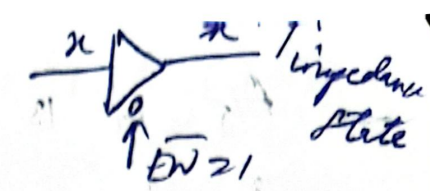
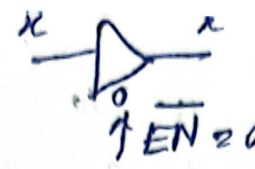
OUT port address (04)

Halt

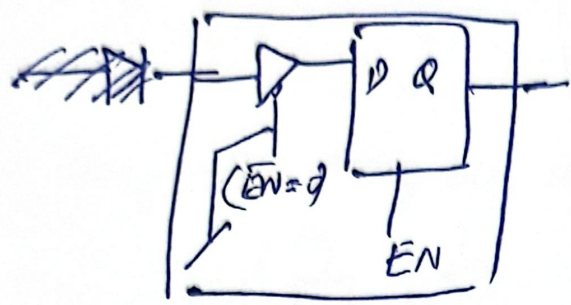
B0H $\rightarrow$ code for 3.

has 6 bits for the port address

Tri-state buffer: 3 conditions $\rightarrow$



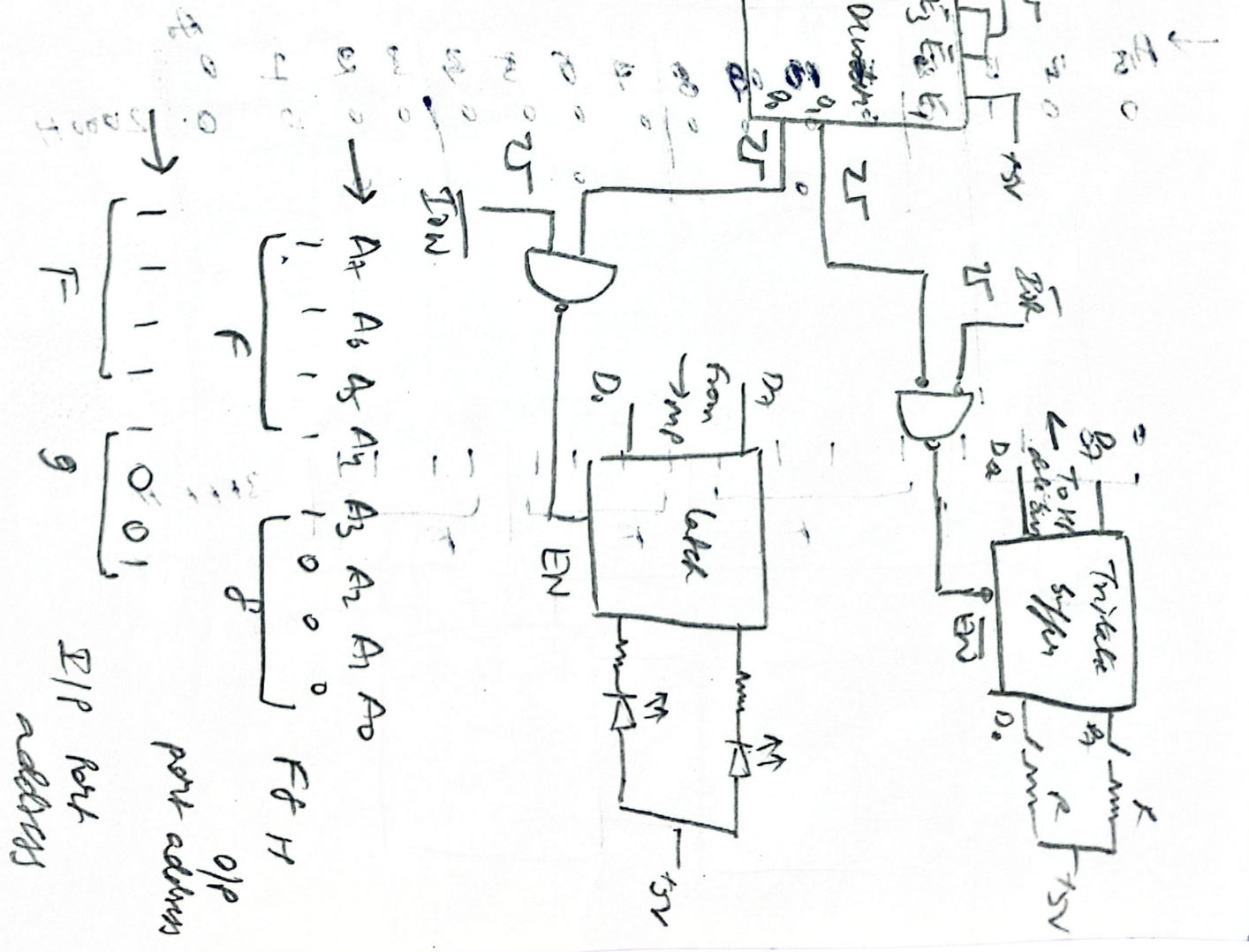
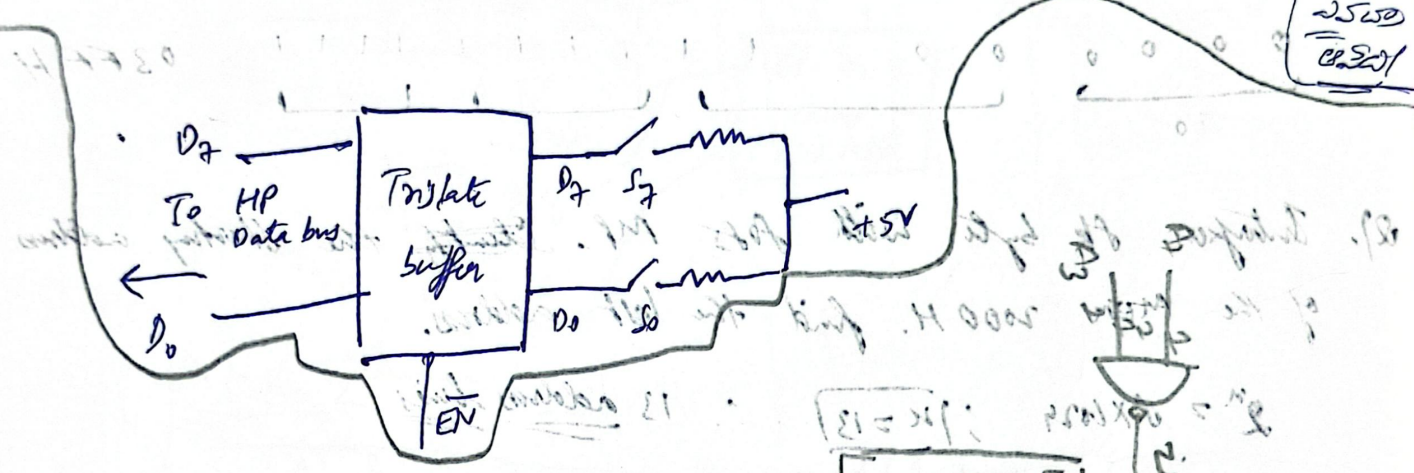
Latch:



Decoder

Tri-state: 74LS13P

74LS244 $\rightarrow$ Buffer



→ if $J_0 = 1, J_3 = 1$

$\begin{matrix} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ \hline L & 9 & 8 & 7 & 6 & 5 & 4 & 3 \end{matrix} \rightarrow \text{displays "H"}$

→ IN F9 H
 ↑
 input address → inside is "H" & it finds 'H' to Accumulator
 OUT F8 H → output port address
 HLT
 ↓
 210 ↔ 011
 900 jump Accumulator

24/10/16 Microprocessor Instruction set

→ Addressing modes of 8085 mp: Technique to specify operand (data) in an instruction is called addressing mode.

1). Direct Addressing mode: operand is in instruction itself

Ex. (i) STA address ← ex STA 2600H
 ↓
 store. data at 2600H is moved to A.
 (ii) IN port address (iii) LDA address
 ↓
 OUT " load accumulator with address

2). Register Addressing mode: operand is one of the register

Ex. MOV A, B ; $A \leftarrow B$

3). Immediate addressing mode: Ex. MOV A, data

4). Indirect addressing mode: LXI H, 2500 (move data at 2500 to HL pair (memory pointer))
 ↓
 load immediately
 MOV A, M

5). Implicit Addressing mode: RAR " " RPT
 RAL → Rotate accumulator left
 CMA → Complement Accumulator

① Data Transfer Instruction

→ MOV R₁, R₂ → R₁ ← R₂

Ex: MOV A, B
MOV B, C

→ MVI R, data → R ← Data (8 bit data)

→ LDA, address

→ STA, address → used to store accumulator content to certain address

→ LHLD, address → Load H, L pair with the content in the address

↑
These 2 are
for 16 bit data
handling
↓

H → MSB, L → LSB

one is address in [L]

next in H+1 " [H]

→ SHLD, address → store the data in H&L in the address
L will be in one address & H " " " (address+1) location.

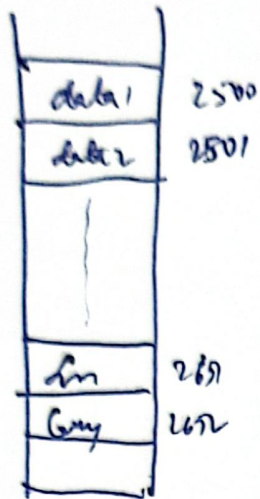
Q). (i) Write an assembly lang. program to add 2-8 bit nos with carry.

Ans: (ii) " " " " " " " " 2-16 bit data with carry.

Ans: (i)
XRA → clear A (primary)
MVI B, Data1
MVI A, Data2
MVI C, 00
ADD B

JNC address
jump with no carry
INR C
Address STA, 2651 (example address)
MOV A, C
STA 2652
HLT

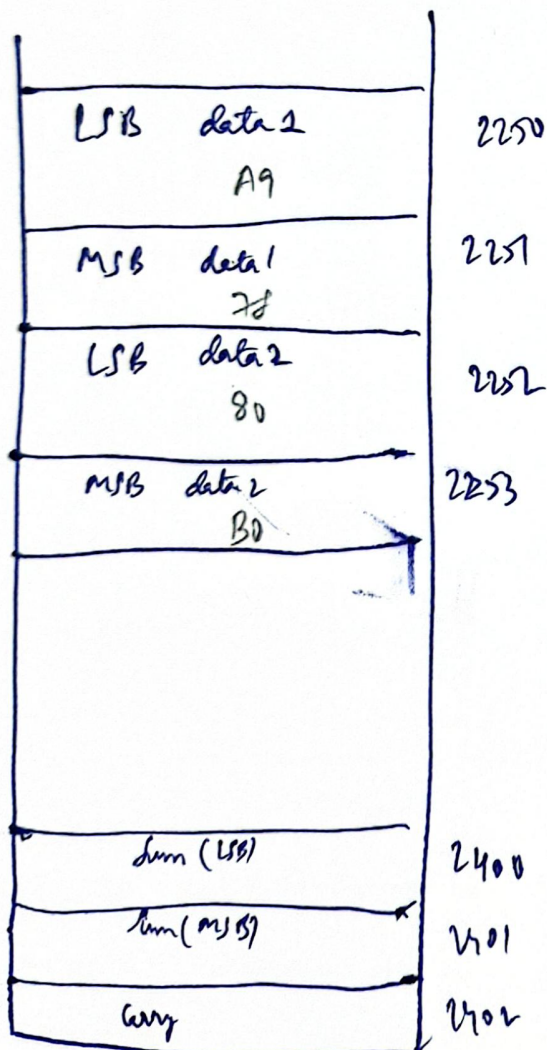
Another method



```

LXI H, 2500
MOV A, M    ( A ← data1 )
INX H      ( M ← data2 )
MVI C, 00   → Carry Status = 00
ADD M      → A ← A + M
JNC add
INR C
Address STA, 2651
MOV A, C
STA 2652
HLT
    
```

(iii) sum of 16 bit with carry



Ex. 78A9
B080

```

LHLD 2250    [L] ← A9 [H] ← 78
XCHG D      → (H-L) ↔ (D-E)
              data in HL exchanged
              to DE pair
LHLD 2252
MVI C, 00    (flag/status)
DAD D        → [H-L] + [D-E] → [H-L]
              add      data      saved in
              HL pair
JNC Add
INR C
Addr SHLD 2400
MOV A, C
STA 2402
HLT
    
```

Q. Write an assembly language program to add 2 8 bit Nos with carry & display the result in decimal form.

Use DAA $\rightarrow$ decimal adjust after A (any)

Q. Write AL program to add 5 8 bit Nos with carry.